

APELLIDOS, NOMBRE _____ DNI _____ nº mesa _____

EJERCICIO 1

PUNTOS: 2

1. (1,5 puntos). Para las siguientes funciones, usar las relaciones $\subset$ y $=$ para ordenar la cota superior de complejidad de menor a mayor, indicando también las que son iguales. Justifica por la propiedad del límite los casos que no sean triviales.

$T(n)$, $O(n+1)$, $O(n^2 \log_2 n)$, $O(n^n)$, $O(2n^2)$, $O(n!)$, $O(3 \cdot 2^n)$, $O(n \log \log n)$, $O(52)$, $O(2^n)$, $O(n \log n)$, $O(2 \log_2 n)$, $O(n(n + \log n))$, $O(1)$, $O(n2^n)$, $O\left(\frac{1}{n}\right)$, $O(n^{n+1})$ siendo $T(n)$ la función:

$$T(n) = \begin{cases} n! & \text{si } n < 5 \\ 3 * T\left(\frac{n}{2}\right) + 4 * T\left(\frac{n}{4}\right) + 3n^2 & \text{si } n > 4 \end{cases}$$

2. (0,5 puntos). Indicar por lo menos 5 ejemplos de algoritmos conocidos que tengan algunos de esos órdenes de complejidad. Por ejemplo, $O(n^3)$, algoritmo clásico de multiplicación de matrices cuadradas de tamaño n .

EJERCICIO 2

PUNTOS: 3

Considera la serie que denominaremos “números J” definida a continuación. El n -ésimo número de J es igual a 3 veces el número $(n-1)$ -ésimo, más 4 veces el $(n-2)$ -ésimo. El primer y el segundo números de J valen 0 y 1, respectivamente. Se pide lo siguiente:

1. (2,25 puntos) Escribir tres posibles implementaciones, **simples y cortas**, para el cálculo del n -ésimo número de J con las siguientes estrategias:
 - a. divide y vencerás recursivo.
 - b. Procedimiento iterativo.
 - c. procedimiento directo, mediante una simple operación aritmética.
2. (0,75 puntos) Realizar una estimación del orden de complejidad de los tres algoritmos del apartado anterior. Comparar los órdenes de complejidad obtenidos, estableciendo una relación de orden entre los mismos.

EJERCICIO 3

PUNTOS: 2

Se tiene un barco de mercancías cuya capacidad de carga es de K toneladas y un conjunto de contenedores $c_1, \dots, c_n$, cuyos pesos respectivos (expresados también en toneladas) son $p_1, \dots, p_n$ y para los que se obtiene un beneficio al ser transportados de $b_1, \dots, b_n$ €, respectivamente. Teniendo en cuenta que la capacidad del barco es menor que la suma total de los pesos de los contenedores:

1. (1punto). Diseña un algoritmo que maximice el número de contenedores cargados.
 - Aplica el algoritmo para $n = 6$; $K = 100$; $p = (10, 5, 7, 20, 60, 35)$; $b = (20, 30, 20, 10, 500, 25)$
 - Analiza su complejidad.
2. (1punto). Diseña un algoritmo que intente maximizar el beneficio obtenido.
 - Aplica el algoritmo para $n = 6$; $K = 100$; $p = (10, 5, 7, 20, 60, 35)$; $b = (20, 30, 20, 10, 500, 25)$
 - Analiza su complejidad.

NOTA: Resuélvelos con el esquema **voraz**, aunque no se garantice la solución óptima. Es necesario marcar en el código propuesto a que corresponde cada parte en el esquema general de un algoritmo voraz (criterio, candidatos, función.....). Si hay más de un criterio posible elegir uno

razonadamente y discutir los otros. Comprobar si el algoritmo garantiza la solución óptima en cada caso (la

APELLIDOS, NOMBRE _____ DNI _____ nº mesa _____

demonstración se puede hacer con un contraejemplo).

EJERCICIO 4

PUNTOS: 3

Dado el siguiente algoritmo de ordenación:

```
/*Procedimiento recursivo para realizar la ordenación por ordenaJunio en el subarray vector[pri..n-1]*/
void ordenaJunio(int vector[], int pri, int n) {
/*encuentra el elemento mínimo en el subarray no ordenado [pri..n-1] y lo cambia por vector[pri] */
    int min = pri;
    for (int j = pri + 1; j < n; j++) {
        /* si vector[j] es menor, entonces es el nuevo mínimo */
        if (vector[j] < vector[min]) {
            min = j;    // actualiza el índice del elemento mínimo
        }
    }
    /* intercambiar el elemento mínimo en el subarray vector[pri..n-1] con vector[pri] */
    Intercambia(vector, min, pri);
    if (pri + 1 < n) {
        ordenaJunio (vector, pri + 1, n);
    }
}

/*Procedimiento para intercambiar valores en dos índices en un array */
void Intercambia(int v[], int i, int j){
    int aux;
    aux=v[i]; v[i]=v[j]; v[j]=aux;
}
```

1. (0,75 puntos) Calcular la complejidad del algoritmo propuesto por el método de la ecuación **característica**.
2. (0,5 puntos) Calcular la complejidad del algoritmo propuesto por el Teorema maestro.
3. (0,75 puntos) Calcular la complejidad del algoritmo propuesto por expansión de recurrencia.
4. (1 punto) Escribir una posible implementación con un procedimiento iterativo. Realizar una estimación del orden de complejidad del algoritmo propuesto y comparar los órdenes de complejidad obtenidos, estableciendo una relación de orden entre los mismos.

NOTAS:

- El procedimiento se invoca con $\text{ordenaJunio}(a, \theta, n)$.
- $\sum_{i=0}^{n-1} a_i = \frac{(a_0 + a_{n-1})n}{2}$
- El Teorema maestro aplicado a $T(n) = aT(n-b) + \Theta(n^k)$ es:

$$T(n) \in \begin{cases} O\left(a^{\frac{n}{b}}\right) & \text{si } a > 1 \\ O(n^{k+1}) & \text{si } a = 1 \\ O(n^k) & \text{si } a < 1 \end{cases}$$